

CORE JAVA PRACTICALS 1-18

1. Bank Account Management System

```
public class BankAccount {

    int accountNumber;

    String holderName;

    double balance;

    BankAccount(int accountNumber, String holderName, double balance) {

        this.accountNumber = accountNumber;

        this.holderName = holderName;

        this.balance = balance;

    }

    public void deposit(double amt) {

        balance += amt;

        System.out.println("Deposited amount: ₹" + amt);

    }

    public void deposit(double amt, String remark) {

        balance += amt;

        System.out.println(holderName + " deposited ₹" + amt + " (" + remark + ")");

    }

    public void withdraw(double amt) {
```

```
        if (balance >= amt) {  
            balance -= amt;  
            System.out.println("Withdrawal amount: ₹" + amt);  
        } else {  
            System.out.println("Insufficient balance");  
        }  
    }  
}  
  
public void displayBalance() {  
    System.out.println("Account Number: " + accountNumber);  
    System.out.println("Holder Name: " + holderName);  
    System.out.println("Balance: ₹" + balance);  
}  
  
public void checkMinBalance() {  
    if (balance < 1000) {  
        System.out.println("Warning: Balance below minimum ₹1000");  
    } else {  
        System.out.println("Sufficient balance");  
    }  
}  
}  
  
public static void main(String[] args) {  
  
    BankAccount acc1 = new BankAccount(1001, "Aryan", 40000);  
    BankAccount acc2 = new BankAccount(1002, "Saloni", 99000);
```

```
BankAccount acc3 = new BankAccount(1003, "Panda", 900);
```

```
acc1.deposit(5000);
```

```
acc2.withdraw(3000);
```

```
acc3.deposit(200, "Fund Transfer");
```

```
System.out.println("\n--- Account Details ---\n");
```

```
acc1.displayBalance();
```

```
acc1.checkMinBalance();
```

```
System.out.println();
```

```
acc2.displayBalance();
```

```
acc2.checkMinBalance();
```

```
System.out.println();
```

```
acc3.displayBalance();
```

```
acc3.checkMinBalance();
```

```
}
```

```
}
```

2. Employee Payroll Calculator

```
public class Employee {

    int empId;

    String empName;

    double basicSalary;

    Employee() {

        empId = 0;

        empName = "Unknown";

        basicSalary = 0;

    }

    Employee(int empId, String empName, double basicSalary) {

        this.empId = empId;

        this.empName = empName;

        this.basicSalary = basicSalary;

    }
```

```
public double calculateSalary() {

    double hra = basicSalary * 0.20;

    double da = basicSalary * 0.10;

    return basicSalary + hra + da;

}


public double calculateSalary(double bonus) {

    return calculateSalary() + bonus;

}


public void displayEmployee() {

    System.out.println("Employee ID: " + empId);

    System.out.println("Employee Name: " + empName);

    System.out.println("Basic Salary: ₹" + basicSalary);

}


public static void main(String[] args) {
```

```
Employee e1 = new Employee(101, "Aryan", 30000);

Employee e2 = new Employee(102, "Saloni", 45000);

Employee e3 = new Employee();

System.out.println("---- Employee 1 ----");

e1.displayEmployee();

System.out.println("Total Salary: ₹" + e1.calculateSalary());

System.out.println();

System.out.println("---- Employee 2 ----");

e2.displayEmployee();

System.out.println("Total Salary with Bonus: ₹" +
e2.calculateSalary(5000));

System.out.println();

System.out.println("---- Employee 3 ----");
```

```
        e3.displayEmployee();

        System.out.println("Total Salary: ₹" + e3.calculateSalary());

    }
}
```

3. University Staff Hierarchy

```
class Person {

    String name;

    int age;

    String contactNo;

    Person(String name, int age, String contactNo) {

        this.name = name;

        this.age = age;

        this.contactNo = contactNo;

    }

    void displayInfo() {

        System.out.println("Name: " + name);

        System.out.println("Age: " + age);

    }
}
```

```
        System.out.println("Contact: " + contactNo);

    }

}

class Employee extends Person {

    String empId;

    String department;

    Employee(String name, int age, String contactNo, String empId, String
department) {

        super(name, age, contactNo);

        this.empId = empId;

        this.department = department;

    }

    void displayEmployeeDetails() {

        displayInfo();

        System.out.println("Employee ID: " + empId);

        System.out.println("Department: " + department);

    }

}
```



```

    }

}

class Faculty extends Employee {

    String designation;

    String[] subjectsTaught;

    Faculty(String name, int age, String contactNo, String empId, String
department,

        String designation, String[] subjectsTaught) {

        super(name, age, contactNo, empId, department);

        this.designation = designation;

        this.subjectsTaught = subjectsTaught;

    }

    void displayFacultyDetails() {

        displayEmployeeDetails();

        System.out.println("Designation: " + designation);

        System.out.print("Subjects: ");

```

```
        for (String subject : subjectsTaught) {

            System.out.print(subject + " ");

        }

        System.out.println();

    }

}
```

```
class HodFaculty extends Faculty {

    double additionalAllowance;

    HodFaculty(String name, int age, String contactNo, String empId, String
department,

                String designation, String[] subjectsTaught, double
additionalAllowance) {

        super(name, age, contactNo, empId, department, designation,
subjectsTaught);

        this.additionalAllowance = additionalAllowance;

    }

    void displayHodDetails() {
```

```
        displayFacultyDetails();

        System.out.println("Additional Allowance: ₹" + additionalAllowance);

    }

}

class AdminStaff extends Employee {

    String role;

    AdminStaff(String name, int age, String contactNo, String empId, String
department, String role) {

        super(name, age, contactNo, empId, department);

        this.role = role;

    }

    void displayAdminDetails() {

        displayEmployeeDetails();

        System.out.println("Role: " + role);

    }

}
```

```
public class Main {

    public static void main(String[] args) {

        String subjects[] = {"Java", "OS"};

        Faculty f1 = new Faculty("Dr. Mehta", 45, "9876543210", "F001", "IT",

            "Professor", subjects);

        HodFaculty hod = new HodFaculty("Dr. Sharma", 50, "9998887776", "H001",

            "IT", "HOD", subjects, 5000);

        AdminStaff admin = new AdminStaff("Mr. Patil", 40, "9123456789",

            "A001", "Administration", "Office Manager");

        System.out.println("---- Faculty Details ----");

        f1.displayFacultyDetails();
```

```
        System.out.println("\n---- HOD Details ----");

        hod.displayHodDetails();

        System.out.println("\n---- Admin Staff Details ----");

        admin.displayAdminDetails();

    }
}
```

} 4. E-Commerce Product Catalogue

```
class Product {

    String productId;

    String name;

    double price;

    Product(String productId, String name, double price) {

        this.productId = productId;

        this.name = name;

        this.price = price;

    }
}
```

```
void displayProduct() {

    System.out.println("Product ID: " + productId);

    System.out.println("Product Name: " + name);

    System.out.println("Price: ₹" + price);

}

}

class Electronics extends Product {

    String brand;

    int warrantyYears;

    Electronics(String productId, String name, double price, String brand, int warrantyYears) {

        super(productId, name, price);

        this.brand = brand;

        this.warrantyYears = warrantyYears;

    }

    @Override
```

```
void displayProduct() {

    super.displayProduct();

    System.out.println("Brand: " + brand);

    System.out.println("Warranty: " + warrantyYears + " years");

}

}

class Clothing extends Product {

    String size;

    String material;

    Clothing(String productId, String name, double price, String size, String
material) {

        super(productId, name, price);

        this.size = size;

        this.material = material;

    }

    @Override
```

```
void displayProduct() {  
  
    super.displayProduct();  
  
    System.out.println("Size: " + size);  
  
    System.out.println("Material: " + material);  
  
}  
  
}
```

```
class Grocery extends Product {  
  
    String expiryDate;  
  
    double weight;  
  
    Grocery(String productId, String name, double price, String expiryDate,  
double weight) {  
  
        super(productId, name, price);  
  
        this.expiryDate = expiryDate;  
  
        this.weight = weight;  
  
    }  
  
    @Override
```



```
void displayProduct() {

    super.displayProduct();

    System.out.println("Expiry Date: " + expiryDate);

    System.out.println("Weight: " + weight + " kg");

}

}

public class Main {

    public static void main(String[] args) {

        Product products[] = {

            new Electronics("E101", "Laptop", 55000, "HP", 2),

            new Electronics("E102", "Mobile", 25000, "Samsung", 1),

            new Clothing("C101", "Shirt", 1200, "L", "Cotton"),

            new Clothing("C102", "Jeans", 1800, "32", "Denim"),

            new Grocery("G101", "Rice", 900, "12-12-2026", 10),

            new Grocery("G102", "Milk", 60, "15-05-2026", 1)

        };
    }
}
```

```
        for (Product p : products) {

            System.out.println("-----");

            p.displayProduct();

        }

    }
}
```

} 5. College Admission Counter

```
class Person {

    String name;

    Person(String name) {

        this.name = name;

    }

}

class Student extends Person {

    int rollNo;

    String branch;
```

```
static int totalAdmitted = 0;
```

```
final int MAX_CAPACITY = 60;
```

```
Student() {
```

```
    this("Unknown", 0, "Not Assigned");
```

```
}
```

```
Student(String name, int rollNo, String branch) {
```

```
    super(name);
```

```
    this.rollNo = rollNo;
```

```
    this.branch = branch;
```

```
    totalAdmitted++;
```

```
}
```

```
static int getTotalAdmitted() {
```

```
    return totalAdmitted;
```

```
}
```

```
void displayStudent() {
```

```
    System.out.println("Name: " + name);
```

```
    System.out.println("Roll No: " + rollNo);
```

```
    System.out.println("Branch: " + branch);
```

```
    System.out.println("Max Capacity: " + MAX_CAPACITY);
```

```
    System.out.println();
```

```
}
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Student s1 = new Student("Aryan", 101, "CS");
```

```
        Student s2 = new Student("Saloni", 102, "IT");
```

```
        Student s3 = new Student("Panda", 103, "ENTC");
```

```
        Student s4 = new Student("Asha", 104, "CS");
```

```
Student s5 = new Student("Raj", 105, "Mechanical");

s1.displayStudent();

s2.displayStudent();

s3.displayStudent();

s4.displayStudent();

s5.displayStudent();


System.out.println("Total Admitted Students: " +
Student.getTotalAdmitted());


// s1.MAX_CAPACITY = 100;

// Compile-time Error because MAX_CAPACITY is final

}
```

6. Vehicle Registration

```
class Vehicle {

    String ownerName;

    String vehicleNumber;
```

```
static int totalVehicles;
```

```
static {
```

```
    totalVehicles = 0;
```

```
    System.out.println("Static Block Executed: Vehicle Registration System  
Started");
```

```
}
```

```
{
```

```
    System.out.println("Instance Block Executed: New Vehicle Object  
Created");
```

```
}
```

```
Vehicle(String ownerName, String vehicleNumber) {
```

```
    this.ownerName = ownerName;
```

```
    this.vehicleNumber = vehicleNumber;
```

```
    totalVehicles++;
```

```
}
```

```
final void displayVehicle() {
```

```
        System.out.println("Owner Name: " + ownerName);

        System.out.println("Vehicle Number: " + vehicleNumber);

    }

    static void showTotalVehicles() {

        System.out.println("Total Registered Vehicles: " + totalVehicles);

    }

}

final class RegistrationOffice {

    void officeInfo() {

        System.out.println("RTO Registration Office");

    }

}

public class Main {

    public static void main(String[] args) {
```

```
Vehicle v1 = new Vehicle("Aryan", "MH30AB1234");
```

```
Vehicle v2 = new Vehicle("Saloni", "MH31CD5678");
```

```
Vehicle v3 = new Vehicle("Panda", "MH29EF9012");
```

```
System.out.println("\nVehicle Details:");
```

```
v1.displayVehicle();
```

```
System.out.println();
```

```
v2.displayVehicle();
```

```
System.out.println();
```

```
v3.displayVehicle();
```

```
System.out.println();
```

```
Vehicle.showTotalVehicles();
```

```
RegistrationOffice r = new RegistrationOffice();
```

```
r.officeInfo();
```



```
}
```

7. Online Payment Gateway

```
class Payment {
```

```
    void processPayment(double amount) {
```

```
        System.out.println("Processing payment of ₹" + amount);
```

```
    }
```

```
    void processPayment(double amount, String couponCode) {
```

```
        double discount = amount * 0.10;
```

```
        amount = amount - discount;
```

```
        System.out.println("Payment after coupon discount: ₹" + amount);
```

```
    }
```

```
}
```

```
class CreditCardPayment extends Payment {
```

```
    @Override
```

```
    void processPayment(double amount) {
```

```
        System.out.println("Credit Card Payment Processed: ₹" + amount);

    }

}

class UpiPayment extends Payment {

    @Override

    void processPayment(double amount) {

        System.out.println("UPI Payment Processed: ₹" + amount);

    }

}

class WalletPayment extends Payment {

    @Override

    void processPayment(double amount) {

        System.out.println("Wallet Payment Processed: ₹" + amount);

    }

}
```

```
public class Main {

    public static void main(String[] args) {

        Payment p1 = new CreditCardPayment();

        Payment p2 = new UpiPayment();

        Payment p3 = new WalletPayment();

        System.out.println("---- Runtime Polymorphism ----");

        p1.processPayment(5000);

        p2.processPayment(2500);

        p3.processPayment(1200);

        System.out.println("\n---- Compile Time Polymorphism ----");

        Payment p = new Payment();

        p.processPayment(4000);

        p.processPayment(4000, "SAVE10");

    }

}
```

} 8. Smart Home Device Control

```
class DeviceController {

    void setTimer(int minutes) {

        System.out.println("Timer set for " + minutes + " minutes");

    }

    void setTimer(int hours, int minutes) {

        System.out.println("Timer set for " + hours + " hour " + minutes + "
minutes");

    }

    void setTimer(String schedule) {

        System.out.println("Timer scheduled at " + schedule);

    }

}

class SmartDevice {

    void activate() {

        System.out.println("Device Activated");

    }

}
```

```

    }

}

class SmartLight extends SmartDevice {

    int brightness;

    SmartLight(int brightness) {

        this.brightness = brightness;

    }

    @Override

    void activate() {

        System.out.println("Turning lights ON at " + brightness + "%
brightness");

    }

}

class SmartAC extends SmartDevice {

    int temperature;

```

```
SmartAC(int temperature) {

    this.temperature = temperature;

}

@Override

void activate() {

    System.out.println("Setting AC to " + temperature + "°C");

}

}

class SmartFan extends SmartDevice {

    int speed;

    SmartFan(int speed) {

        this.speed = speed;

    }

}
```

```
@Override

void activate() {

    System.out.println("Fan speed set to " + speed);

}

}

public class Main {

    public static void main(String[] args) {

        DeviceController dc = new DeviceController();

        System.out.println("---- Compile Time Polymorphism ----");

        dc.setTimer(30);

        dc.setTimer(1, 45);

        dc.setTimer("Morning 6:00 AM");

        SmartDevice devices[] = {

            new SmartLight(80),
```

```
        new SmartAC(22),

        new SmartFan(3)

    };

    System.out.println("\n---- Runtime Polymorphism ----");

    for (SmartDevice d : devices) {

        d.activate();

        if (d instanceof SmartLight)

            System.out.println("Device Type: SmartLight");

        else if (d instanceof SmartAC)

            System.out.println("Device Type: SmartAC");

        else if (d instanceof SmartFan)

            System.out.println("Device Type: SmartFan");
```



```
        System.out.println();

    }

}
```

} 9. Insurance Premium Calculator

```
import java.util.*;
```

```
abstract class InsurancePolicy {
```

```
    String policyNo;
```

```
    String holderName;
```

```
    double sumInsured;
```

```
    InsurancePolicy(String policyNo, String holderName, double sumInsured) {
```

```
        this.policyNo = policyNo;
```

```
        this.holderName = holderName;
```

```
        this.sumInsured = sumInsured;
```

```
    }
```

```
    abstract double calculatePremium();
```

```
void displayPolicy() {

    System.out.println("Policy No: " + policyNo);

    System.out.println("Holder Name: " + holderName);

    System.out.println("Sum Insured: ₹" + sumInsured);

}

}

interface Taxable {

    double applyGST(double premium);

}

class HealthInsurance extends InsurancePolicy implements Taxable {

    HealthInsurance(String policyNo, String holderName, double sumInsured) {

        super(policyNo, holderName, sumInsured);

    }

}
```

```
double calculatePremium() {

    return sumInsured * 0.02;

}

public double applyGST(double premium) {

    return premium + (premium * 0.18);

}

}

class VehicleInsurance extends InsurancePolicy implements Taxable {

    VehicleInsurance(String policyNo, String holderName, double sumInsured) {

        super(policyNo, holderName, sumInsured);

    }

    double calculatePremium() {

        return sumInsured * 0.03;

    }

}
```

```
public double applyGST(double premium) {  
  
    return premium + (premium * 0.18);  
  
}  
  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        ArrayList<InsurancePolicy> policies = new ArrayList<>();  
  
        policies.add(new HealthInsurance("H001", "Asha", 250000));  
  
        policies.add(new HealthInsurance("H002", "Raj", 500000));  
  
        policies.add(new HealthInsurance("H003", "Aryan", 300000));  
  
        policies.add(new VehicleInsurance("V001", "Priya", 150000));  
  
        policies.add(new VehicleInsurance("V002", "Saloni", 400000));  
  
    }  
}
```

```
System.out.println("---- All Policies ----");

for (InsurancePolicy p : policies) {

    p.displayPolicy();

    double premium = p.calculatePremium();

    double finalPremium = ((Taxable)p).applyGST(premium);

    System.out.println("Premium: ₹" + premium);

    System.out.println("Premium with GST: ₹" + finalPremium);

    System.out.println();

}

policies.sort((p1, p2) -> Double.compare(p1.sumInsured,
p2.sumInsured));

System.out.println("---- Sorted by Sum Insured ----");

for (InsurancePolicy p : policies) {

    System.out.println(p.policyNo + " - ₹" + p.sumInsured);

}
```

```

    }

    System.out.println("\n---- Premium > 5000 ----");

    policies.stream()

        .filter(p -> p.calculatePremium() > 5000)

        .forEach(p -> System.out.println(p.policyNo + " : ₹" +
p.calculatePremium()));

    }

```

10. Food Delivery Order

```

import java.util.*;

abstract class FoodItem {

    String itemName;

    double price;

    FoodItem(String itemName, double price) {

        this.itemName = itemName;

        this.price = price;

    }

```

```
abstract void prepareItem();
```

```
void displayItem() {
```

```
    System.out.println("Item: " + itemName);
```

```
    System.out.println("Price: ₹" + price);
```

```
}
```

```
}
```

```
interface Discountable {
```

```
    double applyDiscount(double percent);
```

```
}
```

```
interface Trackable {
```

```
    String trackOrder();
```

```
}
```

```
class VegItem extends FoodItem implements Discountable, Trackable {
```

```
VegItem(String itemName, double price) {

    super(itemName, price);

}

void prepareItem() {

    System.out.println("Preparing Veg: " + itemName);

}

public double applyDiscount(double percent) {

    return price - (price * percent / 100);

}

public String trackOrder() {

    return "Delivery in 20 minutes";

}

}
```



```
class NonVegItem extends FoodItem implements Discountable, Trackable {

    NonVegItem(String itemName, double price) {

        super(itemName, price);

    }

    void prepareItem() {

        System.out.println("Preparing Non-Veg: " + itemName);

    }

    public double applyDiscount(double percent) {

        return price - (price * percent / 100);

    }

    public String trackOrder() {

        return "Delivery in 30 minutes";

    }

}
```

```
public class Main {

    public static void main(String[] args) {

        ArrayList<FoodItem> menu = new ArrayList<>();

        menu.add(new VegItem("Paneer Butter Masala", 320));

        menu.add(new NonVegItem("Chicken Biryani", 280));

        menu.add(new VegItem("Dal Tadka", 180));

        menu.add(new NonVegItem("Mutton Curry", 380));

        System.out.println("---- Order Processing ----");

        for (FoodItem item : menu) {

            item.prepareItem();

            item.displayItem();

            Discountable d = (Discountable)item;
```

```

        Trackable t = (Trackable)item;

        System.out.println("After 10% Discount: ₹" + d.applyDiscount(10));

        System.out.println(t.trackOrder());

        System.out.println();
    }

```

```

menu.sort((a, b) -> Double.compare(a.price, b.price));

```

```

System.out.println("---- Sorted Menu ----");

```

```

menu.forEach(item -> {

```

```

    Discountable d = (Discountable)item;

```

```

    System.out.println(item.itemName + " - ₹" + d.applyDiscount(10));

```

```

});

```

```

}

```

11. Library Management

```

package library;

```

```
public class Book {

    public String title;

    protected String author;

    private double price;

    public Book(String title, String author, double price) {

        this.title = title;

        this.author = author;

        this.price = price;

    }

    public void displayBook() {

        System.out.println("Title: " + title);

        System.out.println("Author: " + author);

        System.out.println("Price: ₹" + price);

    }

}
```

```
        public double getPrice() {  
  
            return price;  
  
        }  
import library.Book;  
  
public class Main {  
    public static void main(String[] args) {  
  
        Book b1 = new Book("Java Programming", "James Gosling", 650);  
        Book b2 = new Book("Data Structures", "Mark Allen", 550);  
  
        System.out.println("---- Book Details ----");  
        b1.displayBook();  
  
        System.out.println();  
  
        b2.displayBook();  
  
        System.out.println("\nAccessing public variable:");  
        System.out.println(b1.title);  
  
        System.out.println("\nAccessing private variable using getter:");  
        System.out.println("Price: ₹" + b1.getPrice());  
  
        // Not allowed:  
        // System.out.println(b1.price);
```

```
// Protected works only inside package / subclass  
// System.out.println(b1.author);  
}  
}
```

} 12. Student Marks File Management

```
import java.io.*;

public class Main {

    public static void main(String[] args) {

        try {

            FileWriter fw = new FileWriter("students.txt");

            BufferedWriter bw = new BufferedWriter(fw);

            bw.write("101 Aryan 85");

            bw.newLine();

            bw.write("102 Saloni 92");

            bw.newLine();

            bw.write("103 Panda 78");

            bw.close();
```

```
System.out.println("Student data written successfully.\n");
```

```
FileReader fr = new FileReader("students.txt");
```

```
BufferedReader br = new BufferedReader(fr);
```

```
String line;
```

```
System.out.println("Reading Student Records:\n");
```

```
while ((line = br.readLine()) != null) {
```

```
    System.out.println(line);
```

```
}
```

```
br.close();
```

```
} catch (IOException e) {
```

```
    System.out.println("Error: " + e.getMessage());
```



```
}  
  
}
```

}13. Employee Record File Handling

```
import java.io.*;  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        try {  
  
            File file = new File("employees.txt");  
  
            FileWriter fw = new FileWriter(file, true);  
  
            BufferedWriter bw = new BufferedWriter(fw);  
  
            bw.write("E101 Aryan 35000");  
  
            bw.newLine();  
  
            bw.write("E102 Saloni 42000");
```

```
bw.newLine();
```

```
bw.write("E103 Panda 28000");
```

```
bw.newLine();
```

```
bw.close();
```

```
System.out.println("Employee records written successfully.\n");
```

```
FileReader fr = new FileReader(file);
```

```
BufferedReader br = new BufferedReader(fr);
```

```
String line;
```

```
System.out.println("Employee Records:\n");
```

```
while ((line = br.readLine()) != null) {
```

```
    System.out.println(line);
```

```
}
```

```
        br.close();

    } catch (IOException e) {

        System.out.println("Error: " + e.getMessage());

    }

}
```

} **14. Student Result Analyzer**

```
import java.io.*;

public class Main {

    public static void main(String[] args) {

        try {

            FileWriter fw = new FileWriter("results.txt");

            BufferedWriter bw = new BufferedWriter(fw);
```

```
bw.write("Aryan 85");
```

```
bw.newLine();
```

```
bw.write("Saloni 92");
```

```
bw.newLine();
```

```
bw.write("Panda 78");
```

```
bw.newLine();
```

```
bw.write("Asha 88");
```

```
bw.close();
```

```
FileReader fr = new FileReader("results.txt");
```

```
BufferedReader br = new BufferedReader(fr);
```

```
String line;
```

```
int total = 0;
```

```
int count = 0;
```

```
int highest = 0;
```

```
String topper = "";
```

```
System.out.println("Student Results:\n");
```

```
while ((line = br.readLine()) != null) {
```

```
    System.out.println(line);
```

```
    String data[] = line.split(" ");
```

```
    String name = data[0];
```

```
    int marks = Integer.parseInt(data[1]);
```

```
    total += marks;
```

```
    count++;
```

```
    if (marks > highest) {
```

```
        highest = marks;
```

```
        topper = name;
```

```
    }
```

```

    }

    br.close();

    double average = (double) total / count;

    System.out.println("\nAverage Marks: " + average);

    System.out.println("Topper: " + topper);

    System.out.println("Highest Marks: " + highest);

} catch (IOException e) {

    System.out.println("Error: " + e.getMessage());

}

}

```

15. Product Inventory

```

import java.io.*;

public class Main {

```

```
public static void main(String[] args) {

    try {

        FileWriter fw = new FileWriter("products.txt");

        BufferedWriter bw = new BufferedWriter(fw);

        bw.write("P101 Laptop 55000");

        bw.newLine();

        bw.write("P102 Mobile 25000");

        bw.newLine();

        bw.write("P103 Keyboard 1500");

        bw.newLine();

        bw.write("P104 Mouse 800");

        bw.close();

        BufferedReader br = new BufferedReader(new
FileReader("products.txt"));
```

```
String line;

String searchProduct = "Mobile";

boolean found = false;


System.out.println("Product Inventory:\n");


while ((line = br.readLine()) != null) {


    System.out.println(line);


    String data[] = line.split(" ");


    if (data[1].equalsIgnoreCase(searchProduct)) {


        found = true;


        System.out.println("\nProduct Found:");


        System.out.println("Product ID: " + data[0]);
```



```

        System.out.println("Name: " + data[1]);

        System.out.println("Price: ₹" + data[2]);

    }

}

br.close();

if (!found) {

    System.out.println("\nProduct not found");

}

} catch (IOException e) {

    System.out.println("Error: " + e.getMessage());

}

}

```

16. Attendance Record

```
import java.io.*;
```

```
public class Main {

    public static void main(String[] args) {

        try {

            FileWriter fw = new FileWriter("attendance.txt");

            BufferedWriter bw = new BufferedWriter(fw);

            bw.write("Aryan Present");

            bw.newLine();

            bw.write("Saloni Absent");

            bw.newLine();

            bw.write("Panda Present");

            bw.newLine();

            bw.write("Asha Present");

            bw.newLine();

            bw.write("Raj Absent");
```

```
bw.close();
```

```
        BufferedReader br = new BufferedReader(new  
        FileReader("attendance.txt"));
```

```
        String line;
```

```
        int present = 0;
```

```
        int absent = 0;
```

```
        System.out.println("Attendance Records:\n");
```

```
        while ((line = br.readLine()) != null) {
```

```
            System.out.println(line);
```

```
            String data[] = line.split(" ");
```

```
            if (data[1].equalsIgnoreCase("Present")) {
```

```
                present++;
```

```

        } else if (data[1].equalsIgnoreCase("Absent")) {

            absent++;

        }

    }

    br.close();

    System.out.println("\nTotal Present: " + present);

    System.out.println("Total Absent: " + absent);

} catch (IOException e) {

    System.out.println("Error: " + e.getMessage());

}

}

```

17. Directory File Listing

```

import java.io.*;

public class Main {

    public static void main(String[] args) {

```

```
File folder = new File("records");
```

```
if (!folder.exists()) {
```

```
    folder.mkdir();
```

```
    try {
```

```
        new File("records/student.txt").createNewFile();
```

```
        new File("records/result.txt").createNewFile();
```

```
        new File("records/data.doc").createNewFile();
```

```
        new File("records/marks.txt").createNewFile();
```

```
    } catch (IOException e) {
```

```
        System.out.println("Error creating files");
```

```
    }
```

```
}
```

```
File files[] = folder.listFiles();
```

```
System.out.println("Text Files in Directory:\n");
```

```
for (File file : files) {
```

```
    if (file.isFile() && file.getName().endsWith(".txt")) {
```

```
        System.out.println(file.getName());
```

```
    }
```

```
}
```

```
}
```

```
}
```

18. Student Result Storage

```
import java.io.*;

public class Main {

    public static void main(String[] args) {

        try {

            File folder = new File("results");

            if (!folder.exists()) {

                folder.mkdir();

            }

            File file = new File("results/results.txt");

            FileWriter fw = new FileWriter(file);

            BufferedWriter bw = new BufferedWriter(fw);

            bw.write("101 Aryan 85");

            bw.newLine();

            bw.write("102 Saloni 92");

            bw.newLine();

            bw.write("103 Panda 78");

            bw.newLine();

            bw.write("104 Asha 88");

            bw.close();

            System.out.println("Results stored successfully.\n");
```

```
    FileReader fr = new FileReader(file);

    BufferedReader br = new BufferedReader(fr);

    String line;

    System.out.println("Student Results:\n");

    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }

    br.close();

} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}

}
```